

Task Management System - Requirements Analysis



1. Purpose / System Goal

The Task Management System helps individuals and teams to plan, organize, assign and track tasks efficiently. It provides a simple way to manage daily work, improve productivity and meet deadlines.



2. Users / Stakeholders

- Individual Users - create and manage their own tasks
- Team Members - collaborate on tasks
- Admin (optional) - manage users and system settings



3. Key Benefits

- Organize work in a structured way
- Improve time management
- Increase team collaboration
- Track progress easily
- Reduce missed deadlines
- Save time and increase productivity

MVP



1. User Management

- Register
- Login / Logout
- Profile Update

Manage user accounts securely with registration, login and profile management.



2. Task Management

- Create Task
- View Task List
- Edit Task
- Delete Task

Create, update, delete and view tasks with details, title and description.



3. Task Status

- To Do
- In Progress
- Done

Track task progress using status: To Do, In Progress and Done.



4. Due Date

- Set Due Date
- View Upcoming Tasks

Set due dates for tasks and view upcoming deadlines.



5. Comments

- Add Comment on Task
- View Comments

Add comments to tasks for discussion and view all comments.



6. Notifications

- Due Date Reminder
- Task Assigned Notification
- Status Change Notification

Get notifications for due dates, task assignments and status changes.



4. Functional Requirements

- User can register a new account.
- User can login and logout.
- User can update their profile information.
- User can create a new task.
- User can view all their tasks.
- User can edit an existing task.
- User can delete a task.
- User can set status for a task (To Do, In Progress, Done).
- User can set a due date for a task.
- System can show upcoming tasks based on due date.
- User can add comments on a task.
- User can view all comments for a task.
- System can send due date reminder notifications.
- System can send task assigned notifications.
- System can send status change notifications.



Notes

This document describes the core (MVP) requirements of the Task Management System. Additional features can be added in future based on user needs.



Important Notes & Industry Best Practices:



Use strong passwords and secure authentication.



Validate all user inputs.



Use HTTPS for all communication.



Protect sensitive data and follow privacy rules.



Perform regular backups and monitor system health.



Task Management System – Entity Thinking



1. What is Entity?

An entity is a real-world object or concept about which we store data in the system. Each entity has its own identity and contains related information.



2. How to Identify Entities?

- Read the requirements carefully
- Find the main objects (nouns)
- Identify what data we need to store
- Identify what data is related to each object
- Avoid storing actions or processes as entities



3. Rules for Good Entities

- Each entity must have a unique identity (Primary Key)
- Each entity represents one thing only
- No duplicate information in one entity
- Entities should be independent
- Easy to understand and maintain

4. Key Entities in Task Management System



User

Represents a person who can use the system.
(Individual user or team member)

Example Data:

- Name
- Email
- Password



Task

Represents a piece of work that needs to be done.

Example Data:

- Title
- Description
- Status



Comment

Represents a comment or discussion on a task.

Example Data:

- Comment Text
- Created At



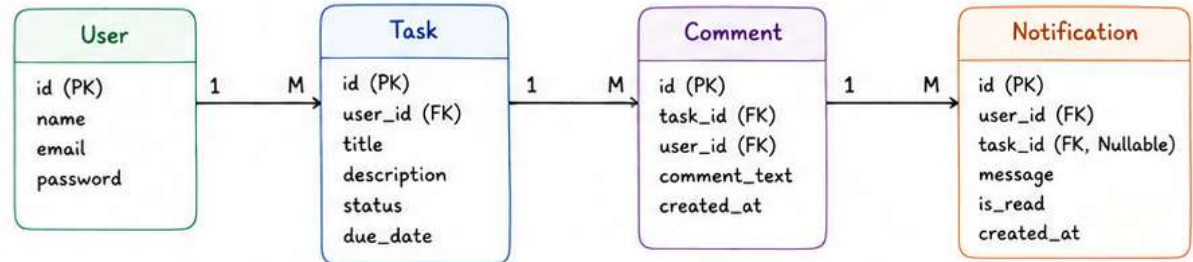
Notification

Represents a message sent to a user about a task or system event.

Example Data:

- Message
- Is Read
- Created At

5. Entity Relationship Overview



Relationship Explanation:

- One User can have Many Tasks, but one Task belongs to one User.
- One Task can have Many Comments, but one Comment belongs to one Task.
- One User can have Many Notifications, but one Notification belongs to one User.
- (Notification can be related to a Task, but it is optional.)



6. Important Points



Focus on data (nouns), not on actions (verbs).



Start with simple entities. Add more if needed later.



Keep entities clear and independent.



Every entity must have a Primary Key.



Good entity design makes the database strong and flexible.



Relationship Diagram Notes



1. What is Relationship?

Relationship defines how entities (tables) are connected with each other in a database.

Example:

- User creates Tasks
- Task contains Comments



2. Why Relationships Important?

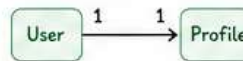
- ✓ Connects tables logically
- ✓ Reduces duplicate data
- ✓ Makes querying easier
- ✓ Maintains data consistency

3. Types of Relationships

1:1 One-to-One

One record connects to only one record.

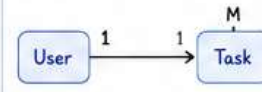
Example:
One User → One Profile



1:M One-to-Many

One record connects to multiple records.

Example:
One User → Many Tasks



Most common relationship.

M:N Many-to-Many

Multiple records connect to multiple records.

Example:
Many Users ↔ Many Teams



Requires a Junction Table.



Relationship Rules



Foreign Key Required

Relationships are created using Foreign Keys.

Example:

tasks.user_id → users.id



Parent → Child Thinking

Parent entity owns the child entity.

Example:

User → Task → Comment



Common Relationship Mistakes

- ✗ Missing Foreign Keys
- ✗ Wrong cardinality (1, M)
- ✗ Duplicate relationships
- ✗ Using Many-to-Many unnecessarily



5. Task Management System Relationships

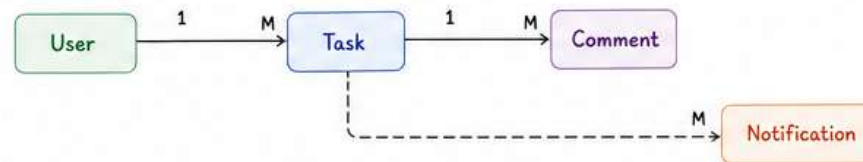
User → Task

One User can create Many Tasks



Task → Comment

One Task can have Many Comments



User → Notification

One User can receive Many Notifications



Symbols

- 1 = One
- M = Many
- = Relationship
- FK = Foreign Key



7. Key Takeaways



Relationships connect tables logically.



Foreign Keys build relationships.



One-to-Many is the most common.



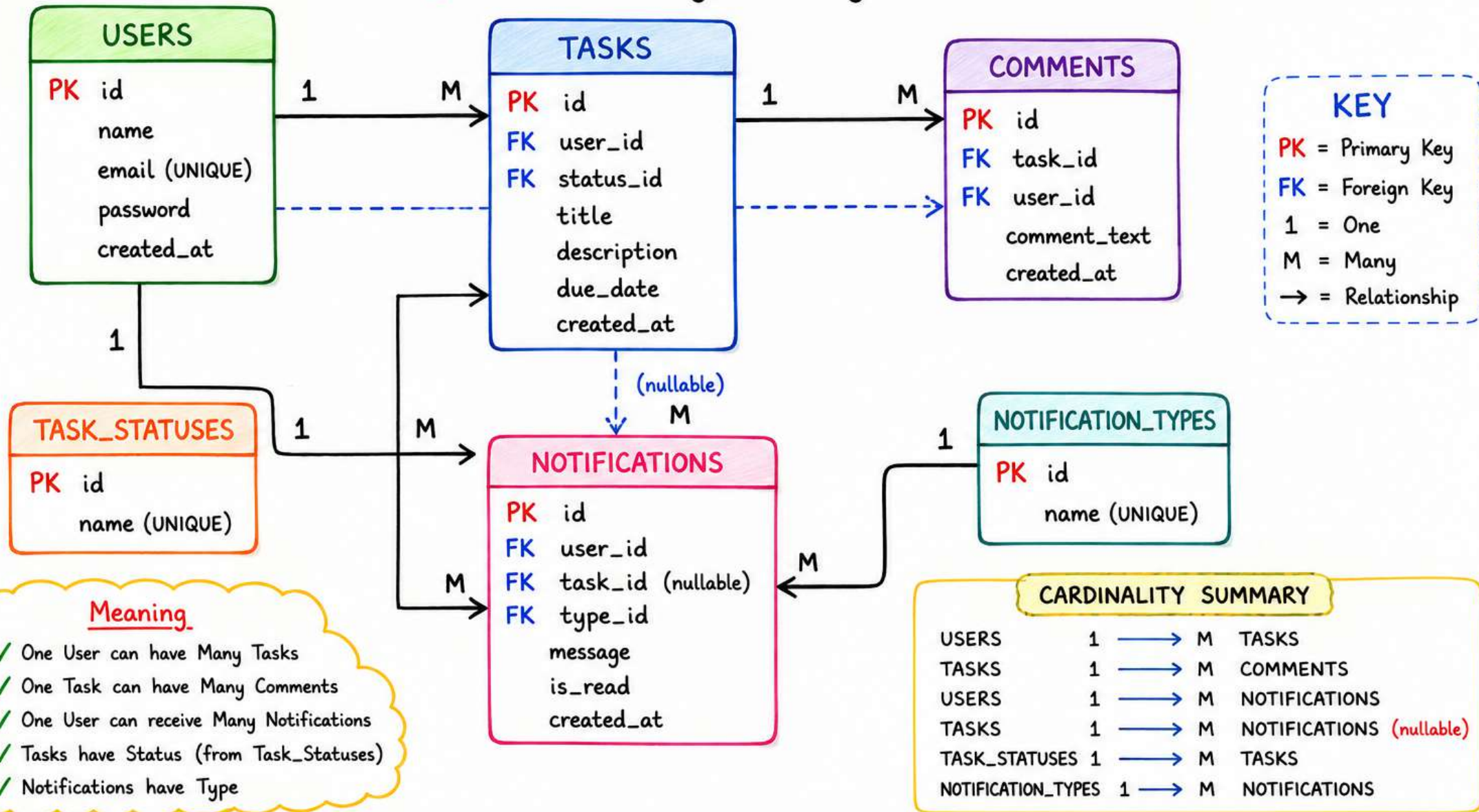
Many-to-Many requires a Junction Table.



Good relationships improve database structure and performance.

RELATIONSHIP DIAGRAM

(Task Management System)



★ Clean Relationships = Strong Database Design ★

ER Diagram HandNotes

1. What is ER Diagram?

ER Diagram (Entity Relationship Diagram) is a visual representation of the database structure.

It shows:

- Entities (Tables)
- Attributes (Columns)
- Relationships
- Primary Keys
- Foreign Keys

2. Purpose

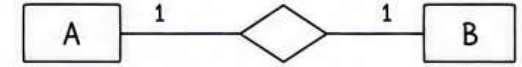
- ✓ Visualize database structure
- ✓ Understand relationships
- ✓ Easy planning & design
- ✓ Improve communication
- ✓ Reduce mistakes

3. Main Components

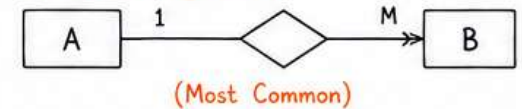
Component	Meaning	Symbol
Entity	Table / Object	
Attribute	Column / Field	
Primary Key (PK)	Unique identifier	<u>Attribute name</u>
Foreign Key (FK)	Links to another table's PK	<u>Attribute name</u>
Relationship	Connection between entities	
Cardinality	1, M, or M:N	1 M M:N

4. Relationship Types

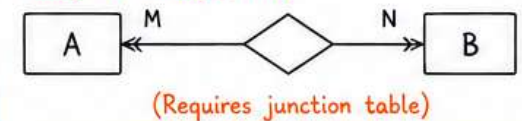
One-to-One (1:1)



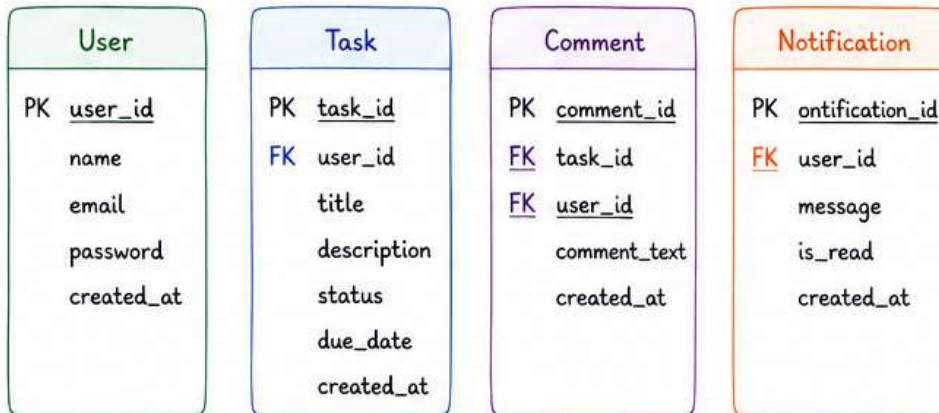
One-to-Many (1:M)



Many-to-Many (M:N)



Task Management System - Entities & Attributes



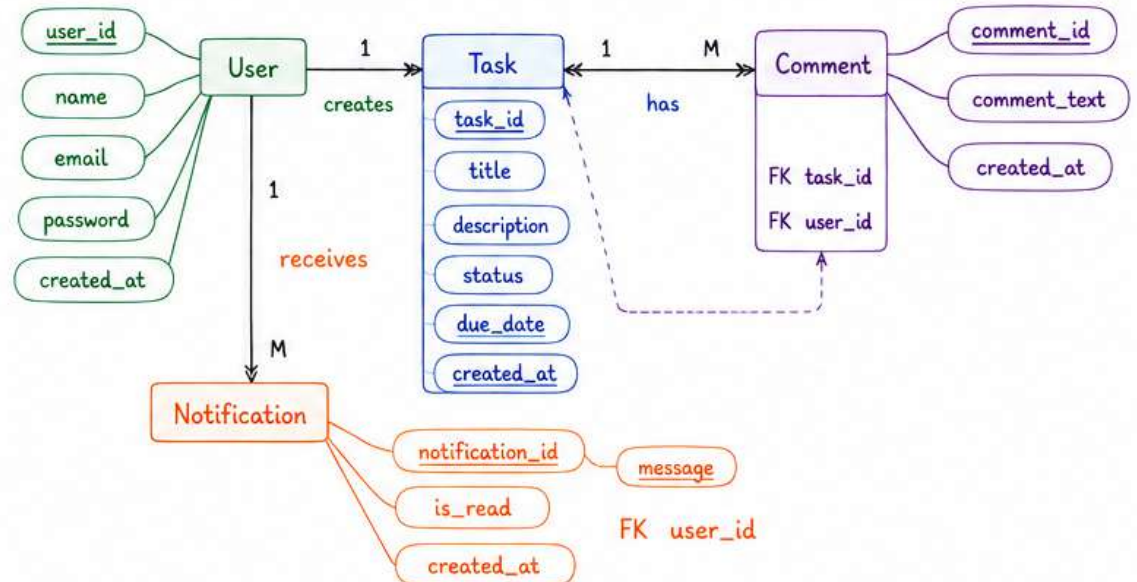
PK = Primary Key

FK = Foreign Key

Underlined = Primary Key

Dashed Underline = Foreign Key

6. Task Management System - ER Diagram



TASK MANAGEMENT SYSTEM – TABLE DESIGN



1. USERS

Column	Type	Key	Constraints	Description
id	BIGSERIAL	PK	NOT NULL	User ID
name	VARCHAR(100)		NOT NULL	User Name
email	VARCHAR(150)		UNIQUE, NOT NULL	User Email
password	VARCHAR(255)		NOT NULL	Hashed Password
created_at	TIMESTAMP		DEFAULT CURRENT_TIMESTAMP	Created Time
updated_at	TIMESTAMP		DEFAULT CURRENT_TIMESTAMP	Last Updated
Indexes: - PRIMARY KEY on id - UNIQUE INDEX on email				



2. TASKS

Column	Type	Key	Constraints	Description
id	BIGSERIAL	PK	NOT NULL	Task ID
user_id	BIGINT	FK →	NOT NULL	Reference to Users
status_id	INT	FK →	NOT NULL	Reference to Task Statuses
title	VARCHAR(255)		NOT NULL	Task Title
description	TEXT			Task Description
due_date	DATE			Due Date
created_at	TIMESTAMP		DEFAULT CURRENT_TIMESTAMP	Created Time
updated_at	TIMESTAMP		DEFAULT CURRENT_TIMESTAMP	Last Updated
Indexes: - INDEX on user_id - INDEX on status_id - INDEX on due_date				



3. TASK_STATUSES (Lookup)

Column	Type	Key	Constraints	Description
id	SERIAL	PK	NOT NULL	Status ID
name	VARCHAR(50)		UNIQUE, NOT NULL	Status Name (To Do, In Progress, Done)
Indexes: - PRIMARY KEY on id - UNIQUE INDEX on name				
Sample Data (1, 'To Do') (2, 'In Progress') (3, 'Done')				



4. COMMENTS

Column	Type	Key	Constraints	Description
id	BIGSERIAL	PK	NOT NULL	Comment ID
task_id	BIGINT	FK →	NOT NULL	Reference to Tasks
user_id	BIGINT	FK →	NOT NULL	Reference to Users
comment_text	TEXT		NOT NULL	Comment Text
created_at	TIMESTAMP		DEFAULT CURRENT_TIMESTAMP	Created Time
Indexes: - INDEX on task_id - INDEX on user_id				



5. NOTIFICATION_TYPES (Lookup)

Column	Type	Key	Constraints	Description
id	SERIAL	PK	NOT NULL	Type ID
name	VARCHAR(100)		UNIQUE, NOT NULL	Type Name (Due Reminder, Task Assigned, Status Changed)
Indexes: - PRIMARY KEY on id - UNIQUE INDEX on name				
Sample Data (1, 'Due Reminder') (2, 'Task Assigned') (3, 'Status Changed')				



6. NOTIFICATIONS

Column	Type	Key	Constraints	Description
id	BIGSERIAL	PK	NOT NULL	Notification ID
user_id	BIGINT	FK →	NOT NULL	Reference to Users (Receiver)
task_id	BIGINT	FK →		Reference to Tasks (Nullable)
type_id	INT	FK →	NOT NULL	Reference to Notification Types
message	TEXT		NOT NULL	Notification Message
is_read	BOOLEAN		DEFAULT FALSE	Read Status
created_at	TIMESTAMP		DEFAULT CURRENT_TIMESTAMP	Created Time
Indexes: - INDEX on user_id - INDEX on type_id - INDEX on type_id				

KEY LEGEND

- PK Primary Key
- FK Foreign Key
- IDX Index

RELATIONSHIP SUMMARY

- USERS 1 —< TASKS
- TASK_STATUSES 1 —< TASKS
- TASKS 1 —< COMMENTS
- USERS 1 —< COMMENTS
- USERS 1 —< NOTIFICATIONS
- TASKS 1 —< NOTIFICATIONS (Optional)
- NOTIFICATION_TYPES 1 —< NOTIFICATIONS

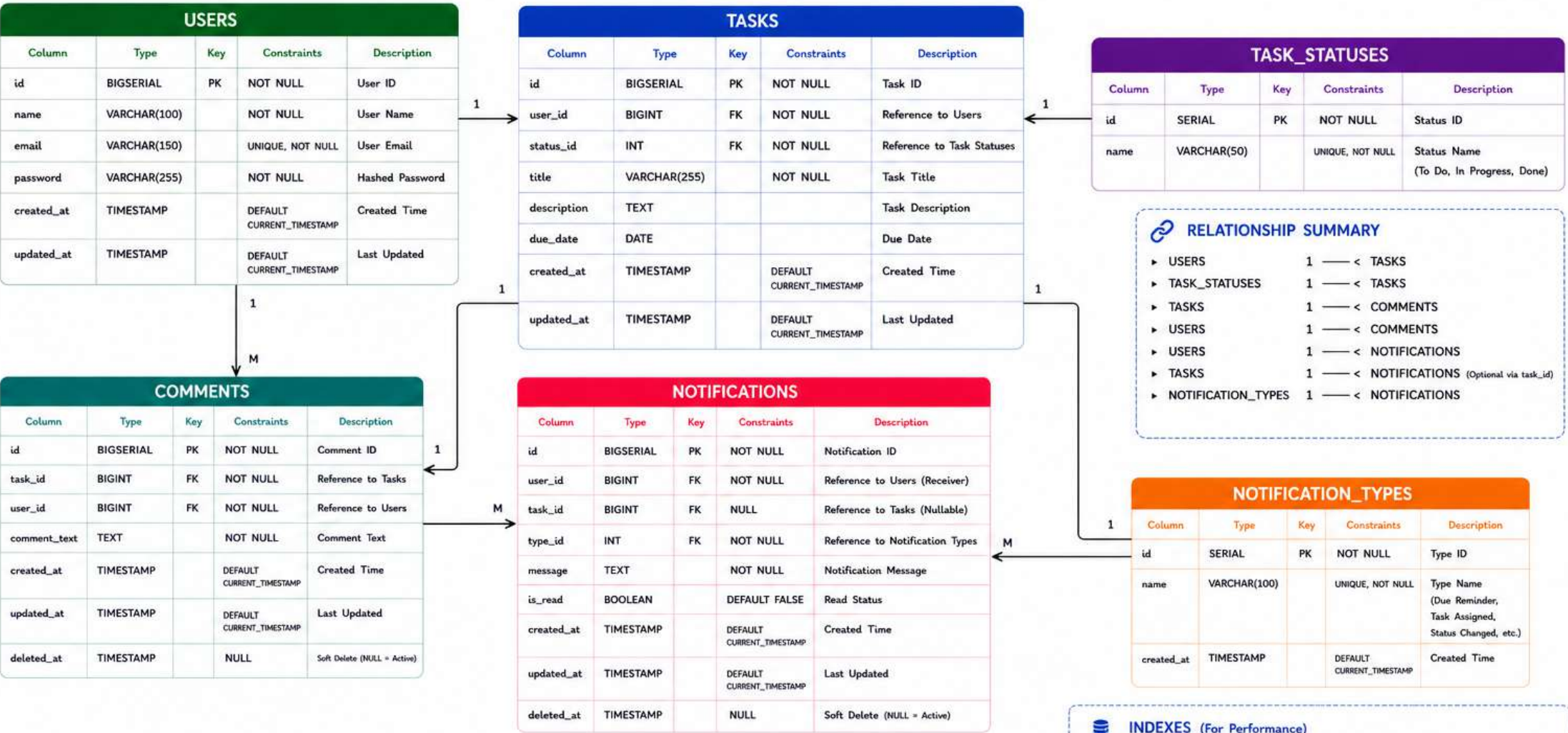
DESIGN NOTES

- ✓ All tables use auto-incrementing keys.
- ✓ Foreign keys enforce referential integrity.
- ✓ ON DELETE CASCADE is used where child data depends on parent.
- ✓ Lookup tables store reusable reference data.
- ✓ All tables follow 3NF (Third Normal Form).

INDEXING STRATEGY

- ✓ Foreign key columns are indexed for better JOIN performance.
- ✓ due_date index helps in task filtering and reporting.
- ✓ is_read index improves unread notification queries.
- ✓ Indexes improve performance but should be used judiciously.

TASK MANAGEMENT SYSTEM – SQL SCHEMA TABLES



DESIGN NOTES

- ✓ All primary keys use BIGSERIAL / SERIAL for auto-increment.
- ✓ Foreign keys ensure referential integrity.
- ✓ ON DELETE CASCADE is used where child data depends on parent.
- ✓ updated_at column is used for tracking changes.
- ✓ Soft delete implemented using deleted_at (optional).
- ✓ All tables follow 3NF (Third Normal Form).



LOOKUP DATA (Sample Inserts)

TASK_STATUSES

- To Do
- In Progress
- Done

NOTIFICATION_TYPES

- Due Reminder
- Task Assigned
- Status Changed

TASK MANAGEMENT SYSTEM – SQL SCHEMA CREATE TABLE STATEMENTS



1. USERS TABLE

```
-- 1. USERS TABLE
CREATE TABLE users (
  id BIGSERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) UNIQUE NOT NULL,
  password VARCHAR(255) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```



2. TASK_STATUSES TABLE (Lookup values)

```
-- 2. TASK_STATUSES TABLE (Lookup values)
CREATE TABLE task_statuses (
  id SERIAL PRIMARY KEY,
  name VARCHAR(50) UNIQUE NOT NULL
);

-- Insert Lookup Values
INSERT INTO task_statuses (name) VALUES
  ('To Do'),
  ('In Progress'),
  ('Done');
```



3. TASKS TABLE

```
-- 3. TASKS TABLE
CREATE TABLE tasks (
  id BIGSERIAL PRIMARY KEY,
  user_id BIGINT NOT NULL,
  status_id INT NOT NULL,
  title VARCHAR(255) NOT NULL,
  description TEXT,
  due_date DATE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_tasks_user
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  CONSTRAINT fk_tasks_status
    FOREIGN KEY (status_id) REFERENCES task_statuses(id)
);
```



4. COMMENTS TABLE

```
-- 4. COMMENTS TABLE
CREATE TABLE comments (
  id BIGSERIAL PRIMARY KEY,
  task_id BIGINT NOT NULL,
  user_id BIGINT NOT NULL,
  comment_text TEXT NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_comments_task
    FOREIGN KEY (task_id) REFERENCES tasks(id) ON DELETE CASCADE,
  CONSTRAINT fk_comments_user
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);
```



5. NOTIFICATION_TYPES TABLE (Lookup values)

```
-- 5. NOTIFICATION_TYPES TABLE (Lookup values)
CREATE TABLE notification_types (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100) UNIQUE NOT NULL
);

-- Insert Lookup Values
INSERT INTO notification_types (name) VALUES
  ('Due Reminder'),
  ('Task Assigned'),
  ('Status Changed');
```



6. NOTIFICATIONS TABLE

```
-- 6. NOTIFICATIONS TABLE
CREATE TABLE notifications (
  id BIGSERIAL PRIMARY KEY,
  user_id BIGINT NOT NULL,
  task_id BIGINT,
  type_id INT NOT NULL,
  message TEXT NOT NULL,
  is_read BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_notifications_user
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  CONSTRAINT fk_notifications_task
    FOREIGN KEY (task_id) REFERENCES tasks(id) ON DELETE CASCADE,
  CONSTRAINT fk_notifications_type
    FOREIGN KEY (type_id) REFERENCES notification_types(id)
);
```



INDEXES

Foreign Key Indexes (Accelerates JOIN operations and cascades)

```
CREATE INDEX idx_tasks_user_id      ON tasks(user_id);
CREATE INDEX idx_tasks_status_id    ON tasks(status_id);
CREATE INDEX idx_comments_task_id   ON comments(task_id);
CREATE INDEX idx_comments_user_id   ON comments(user_id);
CREATE INDEX idx_notifications_user_id ON notifications(user_id);
CREATE INDEX idx_notifications_task_id ON notifications(task_id);
```

Filter Indexes (Improves filtering performance)

```
CREATE INDEX idx_tasks_due_date      ON tasks(due_date);
CREATE INDEX idx_notifications_is_read ON notifications(is_read);
```



Notes:

- BIGSERIAL / SERIAL are used for auto-incrementing primary keys.
- ON DELETE CASCADE is used to automatically delete related records.
- Lookup tables (task_statuses, notification_types) store reusable reference data.
- Indexes improve performance for joins and filtering.

TASK MANAGEMENT SYSTEM – NORMALIZATION REPORT

GOAL : Eliminate redundant data and organize tables to ensure data integrity and consistency.

NORMAL FORM	RULE	CHECKING PROCESS	RESULT (OUR SCHEMA)	REMARKS
1NF	<u>First Normal Form</u> - Each column should have atomic (single) value. - No repeating groups / arrays.	✓ Checked all tables and columns. ✓ All values are atomic. ✓ No repeating groups or multi-valued attributes.	✓ All tables are in 1NF. All values are atomic and each record is unique.	No column contains multiple values. Each column holds single and indivisible value.
2NF	<u>Second Normal Form</u> - Must be in 1NF. - No partial dependency. Every non-key column must be fully dependent on the primary key.	✓ All tables have single column primary key (id). ✓ No table has composite key, so no partial dependency exists.	✓ All tables are in 2NF. Every non-key attribute is fully dependent on the primary key.	Since all tables use single column primary key, partial dependency issue does not exist.
3NF	<u>Third Normal Form</u> - Must be in 2NF. - No transitive dependency. Non-key column should not depend on another non-key column.	✓ Checked for transitive dependency in each table. ✓ All non-key attributes depend only on the primary key. <u>Examples Checked :</u> • users → no transitive dependency • tasks → no transitive dependency • comments → no transitive dependency • notifications → no transitive dependency • lookup tables → no transitive dependency	✓ All tables are in 3NF. No transitive dependency found in any table.	Lookup tables (task_statuses, notification_types) are used to avoid redundancy and transitive dependency.

NORMALIZATION IMPROVEMENTS

- Separated task statuses into task_statuses table.
- Separated notification types into notification_types table.
- This eliminates duplicate values.
- Ensures consistency and easier maintenance.

FINAL RESULT

All tables are normalized up to 3NF.
The database is well-structured, eliminates redundancy, and ensures data integrity.

✓ SCHEMA IS FULLY NORMALIZED

SUMMARY : Our Task Management System database is in 3NF.
It is efficient, consistent, and ready for development.